

Answers and Screenshots — Hello App

Тестовое задание — ответы на вопросы

1) Чем git pull отличается от git fetch?

- ``git fetch`` загружает обновления с удалённого репозитория в локальные remote-tracking ветки без изменения рабочей ветки. - ``git pull`` выполняет ``fetch`` и затем автоматически объединяет изменения в текущую ветку (обычно через ``merge`` или ``rebase``).

2) Soft link vs hard link в Linux

- Hard link — ещё одно имя для того же inode; файл остаётся доступен пока есть хотя бы одна ссылка; нельзя ссылаться на директорию и нельзя через разные файловые системы. - Soft (symbolic) link — файл, содержащий путь; работает через ФС и для директорий; при удалении объекта ссылка становится «висячей».

3) Как проверить сетевую доступность между двумя машинами

- Использовать ``ping`` (ICMP) для базовой проверки. - Проверить конкретный TCP-порт: ``nc -zv HOST PORT`` или ``telnet HOST PORT``. - Проследить маршрут: ``traceroute`/`mtr``. - Для сервисов HTTP: ``curl -I http://HOST:PORT``.

4) На каких механизмах Linux основана контейнеризация в Docker

- Основные механизмы: namespaces (PID, NET, MNT, UTS, IPC, USER), cgroups (ограничение ресурсов), файловые системы с copy-on-write (overlayfs), seccomp, AppArmor/SELinux, capabilities.

5) Почему сначала ``COPY package.json`` → ``npm install`` → ``COPY . .``

- Это позволяет Docker использовать кэш слоёв: установка зависимостей выполняется только при изменении ``package.json``/lock-файла, ускоряя повторные сборки.

6) Могут ли два контейнера в одном Pod слушать один и тот же порт?

- Нет: контейнеры в одном Pod разделяют один network namespace и IP, поэтому порты конфликтуют. Для этого размещают контейнеры в разных Pod'ах.

7) Какие виды JOIN существуют?

- ``INNER JOIN``, ``LEFT JOIN`` (LEFT OUTER), ``RIGHT JOIN`` (RIGHT OUTER), ``FULL JOIN`` (FULL OUTER), ``CROSS JOIN``, ``SELF JOIN``.

8) Что такое HAVING и чем отличается от WHERE?

- ``WHERE`` фильтрует строки до агрегации (не может использовать агрегатные функции). - ``HAVING`` фильтрует после ``GROUP BY`` и может использовать агрегаты.

См. также прикреплённые скриншоты для доказательства рабочих шагов и команды в проекте.

Ссылки: - Репозиторий: <https://github.com/Imronaxl/prototype7> - Схема (draw.io):

https://drive.google.com/file/d/1gJtV93H90MSnVCqSozx_Lxrzv9WVEdt-/view?usp=sharing

```
> docker build -t imeon/hello-app:1.0.0 .
[+] Building 53.7s (11/11) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 464B                                0.0s
=> [internal] load metadata for docker.io/library/python:3.12-slim 0.0s
=> [internal] load .dockerignore                                   0.0s
=> => transferring context: 118B                                     0.0s
=> [1/6] FROM docker.io/library/python:3.12-slim                 0.1s
=> [internal] load build context                                   0.0s
=> => transferring context: 541B                                     0.0s
=> [2/6] RUN groupadd --gid 10001 app && useradd --uid 10001 --gid app --create-home 0.3s
=> [3/6] WORKDIR /app                                              0.0s
=> [4/6] COPY requirements.txt .                                    0.0s
=> [5/6] RUN pip install --no-cache-dir --upgrade pip && pip install --no-cache-dir 52.8s
=> [6/6] COPY app.py .                                             0.0s
=> exporting to image                                              0.3s
=> => exporting layers                                              0.2s
=> => writing image sha256:b6724b0e28cc3a0b2914a9411438cf94554bf0dd1df85e6904d90fad048 0.0s
=> => naming to docker.io/imeon/hello-app:1.0.0                   0.0s
~/biocad/prototype7 main !1 ..... 54s
```



```
> docker push imeon/hello-app:1.0.0
The push refers to repository [docker.io/imeon/hello-app]
8d356bd05210: Layer already exists
85fdd57297cf: Layer already exists
b47d430d4caa: Layer already exists
44ae71e8486e: Pushed
64ff01884ef8: Layer already exists
24257af4ce5b: Layer already exists
3d664d274420: Layer already exists
3070bb8623db: Layer already exists
411a86676185: Layer already exists
1.0.0: digest: sha256:42528fdf24f1a304d829cdddfb5fcbf657da71a034629d894c5c6f7744aecf24 size: 198
~/biocad/prototype7 main !2 ..... 41s
>
```

Repositories / hello-app / General

imeon/hello-app

Last pushed 1 minute ago · ☆0 · ↓0

Add a description

Add a category

General

Tags

Image Management

Collaborators

Webhooks

Settings

Tags

This repository contains 1 tag(s).

Tag	OS	Type	Pulled	Pushed
1.0.0		Image	less than 1 day	1 minute

See all

Repository overview

An overview describes what your image does and how to run it. It displays in the public view of your repository once you have pushed some content.

Add overview

Using 0 of 1 private repositories.

Docker commands

To push a new tag to this repository:

docker push imeon/hello-app:tagname

buildcloud

Build with Docker Build Cloud

Accelerate image build times with access to cloud-based builders and shared cache.

Docker Build Cloud executes builds on optimally-dimensioned cloud infrastructure with dedicated per-organization isolation.

Get faster builds through shared caching across your team, native multi-platform support, and encrypted data transfer - all without managing infrastructure.

Go to Docker Build Cloud

docker/imeon/hello-app/general


```
• > docker run -d -p 32777:32777 --name hello-app imeon/hello-app:1.0.0
```

```
d58dd45f5be6673cf5aa6070fbb5ce5878457554d7980a25bf107155e4c17b8b
```

```
• > curl http://localhost:32777
```

```
{"hostname":"d58dd45f5be6","message":"Hello, World!","port":32777}
```

```
○ [~/biocad/prototype7 main !8 ?2 .....  
>
```



```
> minikube start
🐳 minikube v1.39.0 на Ubuntu 25.10
🌟 Используется драйвер docker на основе существующего профиля
👍 Starting "minikube" primary control-plane node in "minikube" cluster
🚢 Pulling base image v0.0.51 ...
🔄 Обновляется работающий docker "minikube" container ...

🔥 Docker is nearly out of disk space, which may cause deployments to fail! (88% of capacity). You
can pass '--force' to skip this check.
💡 Предложение:

Try one or more of the following to free up space on the device:

1. Run "docker system prune" to remove unused Docker data (optionally with "-a")
2. Increase the storage allocated to Docker for Desktop by clicking on:
   Docker icon > Preferences > Resources > Disk Image Size
3. Run "minikube ssh -- docker system prune" if using the Docker container runtime
🍷 Related issue: https://github.com/kubernetes/minikube/issues/9024

📦 Подготавливается Kubernetes v1.37.0 на containerd 2.3.4 ...
🔍 Компоненты Kubernetes проверяются ...
  ■ Используется образ gcr.io/k8s-minikube/storage-provisioner:v5
🌟 Включенные дополнения: storage-provisioner, default-storageclass

❗ /usr/local/bin/kubectl is version 1.31.0, which may have incompatibilities with Kubernetes 1.37
.0.
  ■ Want kubectl v1.37.0? Try 'minikube kubectl -- get pods -A'
🚀 Готово! kubectl настроен для использования кластера "minikube" и "default" пространства имён по
умолчанию
> minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured

~/biocad/prototype7 main !8 ?3
```

```
~/biocad/prototype7 main !8 ?3 .....  
• > kubectl get nodes  
  
NAME          STATUS    ROLES          AGE    VERSION  
minikube      Ready     control-plane   153m   v1.37.0  
~/biocad/prototype7 main !8 ?4 .....  
○ > 
```



```
> kubectl get pods -l app=hello-app -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED
hello-app-56d95798dc-2h7sq	1/1	Running	0	148m	10.244.0.5	minikube	<none>
hello-app-56d95798dc-4mwcx	1/1	Running	0	148m	10.244.0.4	minikube	<none>


```
> kubectl get svc hello-app
NAME      TYPE        CLUSTER-IP    EXTERNAL-IP  PORT(S)    AGE
hello-app ClusterIP    10.100.47.227 <none>       80/TCP     149m
~/biocad/prototype7 main !8 ?6
```

```
> kubectl port-forward svc/hello-app 8080:80
```

```
Forwarding from 127.0.0.1:8080 -> 32777
```

```
Forwarding from [::1]:8080 -> 32777
```

```
Handling connection for 8080
```

```
Handling connection for 8080
```

```
Handling connection for 8080
```

```
^C
```


Автоформатировать ☐

```
{"hostname":"hello-app-56d95798dc-2h7sq","message":"Hello, World!","port":32777}
```



```
• > curl -H "Host: site2.local" http://localhost:8080/

Handling connection for 8080
{"hostname":"hello-app-56d95798dc-2h7sq","message":"Hello, World!","port":32777}
• > curl -H "Host: site1.local" http://localhost:8080/

Handling connection for 8080
{"hostname":"hello-app-56d95798dc-2h7sq","message":"Hello, World!","port":32777}
~/.biocad/prototype7 main !8 ?9
>
```